



INGENIERÍA EN AUTOMATIZACIÓN Y ROBOTICA

ASIGNATURA: MICROCONTROLADORES

CODIGO: AAUT103

UNIDAD DE APRENDIZAJE: LCD, TECLADOS MATRICIALES, INTERRUPCIONES Y TIMERS.

CRITERIO DE EVALUACIÓN: Evaluar y seleccionar diferentes funciones de librerías pre-definidas, así como fórmulas de cálculo de tiempos de desborde de timers, para la programación e implementación de aplicaciones que empleen pantallas LCD, teclados matriciales y/o interrupciones varias.

TRABAJO COLABORATIVO N°4

MICROCONTROLADORES

Señales Analógicas

- ALUMNOS:**
- María José Córdova
(20.669.897-7)
 - Alex Díaz
(20.807.586-1)

TRABAJO COLABORATIVO N°4

1. OBJETIVOS:

- Reconocer programación en Lenguaje C, Simulación y Montaje para microcontroladores.
- Manejo de entradas analógicas.

2. PASOS PREVIOS:

Revisar y Comprender las Clases para realizar este trabajo colaborativo.

3. INICIO DE LA ACTIVIDAD:

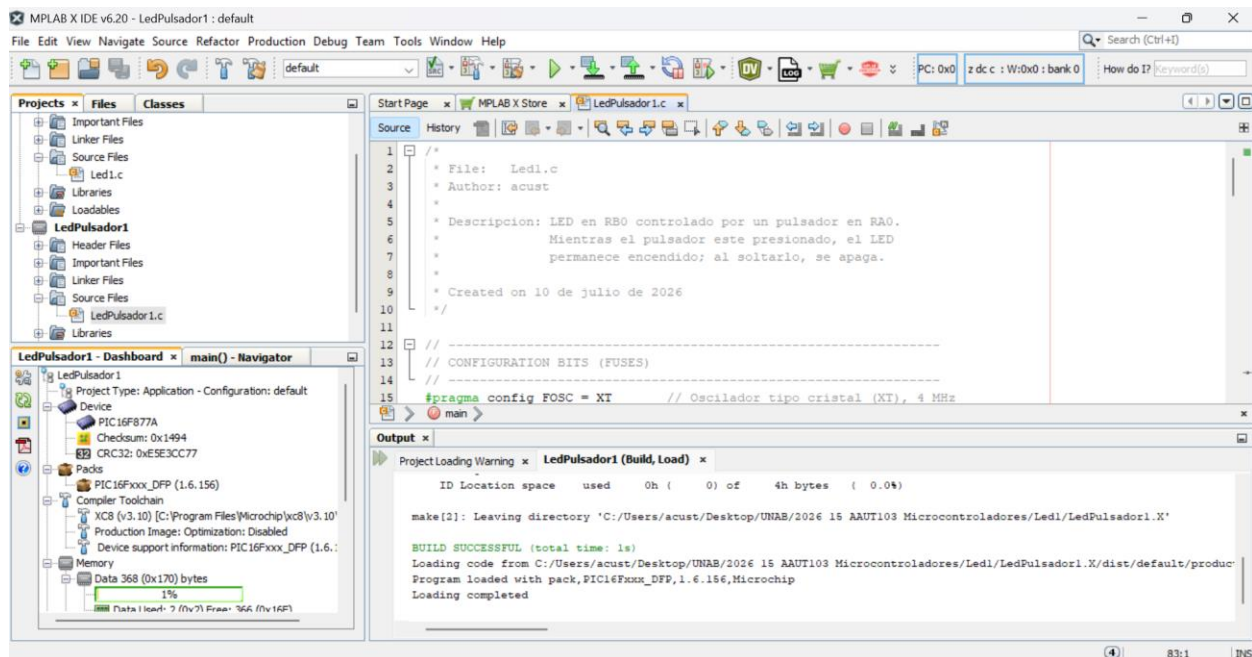
Se realizarán las siguientes actividades.

3.1. INSTRUCCIONES:

En equipo de trabajo establecido se realizarán las siguientes actividades, revisando las exposiciones teóricas y respondiendo las preguntas planteadas. Las actividades (montaje físico) deben estar validadas por el profesor para ser posteriormente evaluadas, luego suba este documento con sus respectivas respuestas al ambiente de aprendizaje.

3.2. PREPARACIÓN PARA LAS ACTIVIDADES DE LABORATORIO:

Revisar la clase sobre el manejo de MPLAB X y el compilador XC8 para PIC



Se utilizará el software PROTEUS o SIMULID para realizar simulaciones del Microcontrolador PIC 16f877a.

3.3. ACTIVIDAD 1: Termómetro Digital con el LM35. (7 puntos)

1. ¿Cómo funciona el LM35?

Es un sensor de temperatura que entrega un voltaje **proporcional linealmente** a la temperatura: **10 mV por cada grado Celsius**. Por ejemplo, a 25°C entrega 250 mV (0.25V); a 100°C entrega 1V. No necesita calibración ni fórmulas complejas — es directo.

2. ¿Cómo convertimos ese voltaje a un número que el PIC entienda?

El PIC16F877A tiene un **módulo ADC (Convertor Analógico-Digital) de 10 bits**, es decir, convierte cualquier voltaje entre 0V y 5V (Vref) en un número entre **0 y 1023**.

$$\text{Valor ADC} = \frac{V_{\text{entrada}}}{5V} \times 1023$$

3. ¿Qué pin usamos?

Vamos a conectar el LM35 a **RA0 (AN0)** — el mismo pin donde antes tenías el pulsador. Como ya vimos con ADCON1, este pin es compartido entre digital y analógico; ahora sí necesitamos que sea **analógico de verdad** (a diferencia de antes, que lo poníamos digital para el pulsador).

4. Configuración de registros nuevos (ADCON0 y ADCON1):

- **ADCON1 = 0x8E**: configura AN0 como analógico (el resto como digital), el resultado del ADC **justificado a la derecha** (más fácil de leer), y el voltaje de referencia como VDD/VSS (5V y 0V).
- **ADCON0 = 0x41**: selecciona el canal AN0, configura el reloj de conversión, y enciende el módulo ADC (ADON = 1).

5. Fórmula para convertir el valor ADC directamente a grados Celsius:

Combinando la fórmula del ADC con la del LM35 (10mV/°C), simplificamos matemáticamente a:

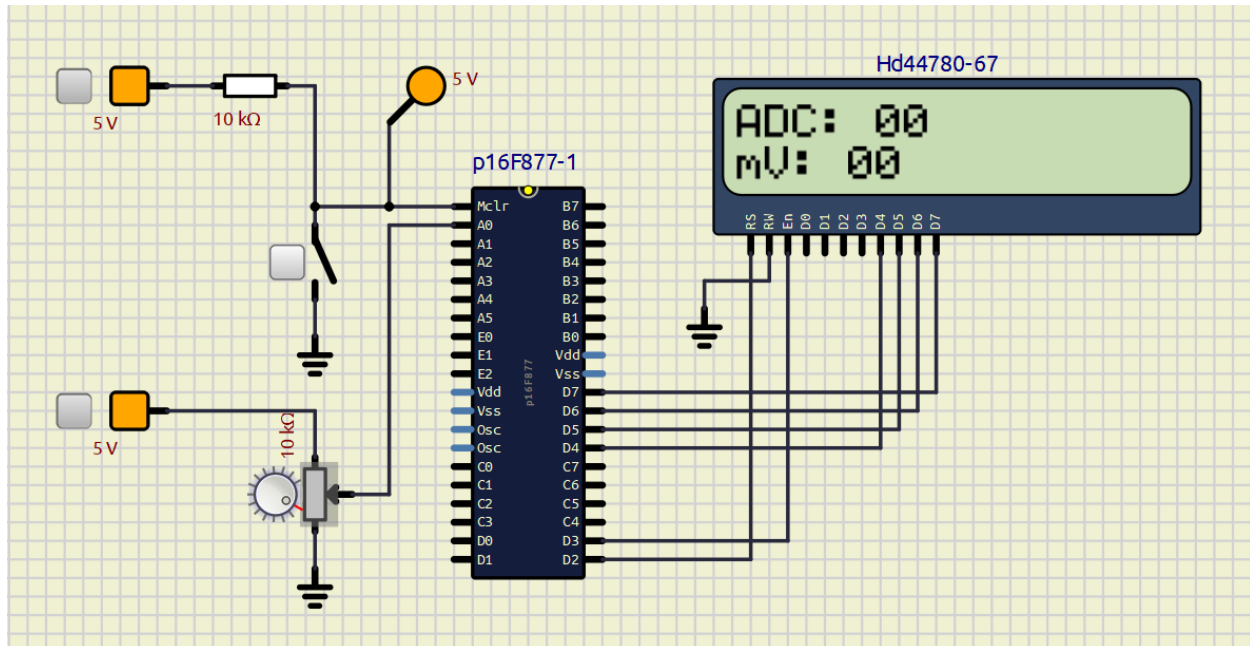
$$\text{Temperatura (°C)} = \frac{\text{Valor ADC} \times 500}{1023}$$

Para simular en SimulID usaremos un potenciómetro:

Pin del potenciómetro	Conexión
Terminal 1	+5V
Terminal 2 (cursor/wiper)	RA0 (AN0) del PIC

Pin del potenciómetro	Conexión
Terminal 3	GND

Usa un potenciómetro de **10kΩ** (valor típico y estándar en estos ejercicios).



En el MPLAB compilar y generar el archivo .hex del siguiente programa:

```

/*
* File: LCD_ADC_Potenciometro.c
* Author: acust
*
* Descripcion: Lee una entrada analogica en RA0 (AN0) mediante
*              un potenciómetro (reemplazo temporal del sensor
*              LM35 para pruebas en simulacion) y muestra en el
*              LCD 16x2 tanto el valor crudo del ADC (0-1023)
*              como el voltaje equivalente en milivoltios (0-5000).
*
* Conexion LCD (todo en PORTD):
* RS -> RD2   E -> RD3   R/W -> GND (fijo)
* D4 -> RD4   D5 -> RD5   D6 -> RD6   D7 -> RD7
*
* Conexion Potenciometro (10k):
* Terminal 1 -> +5V
* Terminal 2 (cursor/wiper) -> RA0 (AN0)
* Terminal 3 -> GND
  
```

*

* Created on 12 de julio de 2026

*/

```
// -----
// CONFIGURATION BITS (FUSES)
// -----
#pragma config FOSC = XT      // Oscilador tipo cristal (XT), 4 MHz
#pragma config WDTE = OFF     // Watchdog Timer deshabilitado
#pragma config PWRTE = ON     // Power-up Timer habilitado
#pragma config BOREN = ON     // Brown-out Reset habilitado
#pragma config LVP = OFF      // Programacion de bajo voltaje deshabilitada
#pragma config CPD = OFF      // Proteccion de memoria de datos deshabilitada
#pragma config WRT = OFF      // Proteccion de escritura en Flash deshabilitada
#pragma config CP = OFF       // Proteccion de codigo deshabilitada

// -----
// LIBRERIAS
// -----
#include <xc.h>

#define _XTAL_FREQ 4000000    // Cristal de 4 MHz

// -----
// DEFINICION DE PINES DEL LCD (todos en PORTD)
// -----
#define LCD_RS    PORTDbits.RD2
#define LCD_EN    PORTDbits.RD3

// -----
// FUNCIONES DEL LCD
// -----

// Pulso de Enable: el LCD "lee" los datos en el flanco de bajada de E
void LCD_Pulso(void) {
    LCD_EN = 1;
    __delay_us(50);
    LCD_EN = 0;
    __delay_us(50);
}

// Envia un nibble (4 bits) por D4-D7 (nibble alto de PORTD)
void LCD_EnviarNibble(unsigned char nibble) {

    // Limpiamos SOLO el nibble alto de PORTD (RD4-RD7).
    // No afecta RD2 (RS) ni RD3 (E), que estan en el nibble bajo.
```

// Colocamos el nuevo nibble en la parte alta del puerto
PORTD |= (nibble << 4);

LCD_Pulso();
}

// Envia un comando al LCD (ej: limpiar pantalla, posicionar cursor)
void LCD_Comando(unsigned char comando) {

LCD_RS = 0; // RS = 0 -> modo COMANDO

LCD_EnviarNibble(comando >> 4); // Nibble alto primero
LCD_EnviarNibble(comando & 0x0F); // Luego nibble bajo

__delay_ms(2); // Tiempo de procesamiento del LCD
}

// Envia un dato (caracter) para mostrar en pantalla
void LCD_Dato(unsigned char dato) {

LCD_RS = 1; // RS = 1 -> modo DATO (caracter)

LCD_EnviarNibble(dato >> 4);
LCD_EnviarNibble(dato & 0x0F);

__delay_ms(2);
}

// Inicializacion del LCD (secuencia obligatoria del fabricante)
void LCD_Inicializar(void) {

__delay_ms(20); // Espera a que el LCD encienda estable

// Secuencia especial de arranque en modo 4 bits (datasheet HD44780)

LCD_EnviarNibble(0x03);

__delay_ms(5);

LCD_EnviarNibble(0x03);

__delay_ms(5);

LCD_EnviarNibble(0x03);

__delay_ms(5);

LCD_EnviarNibble(0x02); // Confirma modo 4 bits

LCD_Comando(0x28); // Modo 4 bits, 2 lineas, fuente 5x8

LCD_Comando(0x0C); // Display ON, cursor OFF, sin parpadeo

```
LCD_Comando(0x06); // Modo de incremento automatico del cursor
LCD_Comando(0x01); // Limpia pantalla
__delay_ms(2);
}

// Escribe una cadena de texto completa
void LCD_EscribirTexto(const char *texto) {
    while (*texto != '\0') {
        LCD_Dato(*texto);
        texto++;
    }
}

// Muestra un numero entero (0-9999) en el LCD, digito por digito
void LCD_EscribirNumero(unsigned int numero) {

    unsigned char miles, centenas, decenas, unidades;

    miles  = numero / 1000;
    centenas = (numero / 100) % 10;
    decenas  = (numero / 10) % 10;
    unidades = numero % 10;

    // Solo mostramos los digitos superiores si son distintos de cero
    // (evita mostrar "0025" en vez de "25")
    if (miles > 0) {
        LCD_Dato(miles + '0');
    }
    if (miles > 0 || centenas > 0) {
        LCD_Dato(centenas + '0');
    }

    LCD_Dato(decenas + '0');
    LCD_Dato(unidades + '0');
}

// -----
// CONFIGURACION Y LECTURA DEL MODULO ADC
// -----

void ADC_Inicializar(void) {

    // ADCON1: ADFM=1 (resultado justificado a la derecha),
    // PCFG3:PCFG0 = 1110 (AN0 analogico, AN1-AN7 digitales,
    // Vref+ = VDD, Vref- = VSS)
    ADCON1 = 0x8E;
```

```
// ADCON0: ADCS1:ADCS0 = 01 (reloj de conversion Fosc/8),  
// CHS2:CHS0 = 000 (canal AN0 seleccionado), ADON = 1 (ADC encendido)  
ADCON0 = 0x41;  
}
```

```
unsigned int ADC_Leer(void) {
```

```
    // Tiempo de adquisicion: esperamos a que el capacitor interno  
    // de muestreo se cargue con el voltaje de entrada  
    __delay_us(20);
```

```
    // Iniciamos la conversion  
    ADCON0bits.GO = 1;
```

```
    // Esperamos a que el bit GO baje a 0 (conversion terminada)  
    while (ADCON0bits.GO == 1) {  
        // No hacemos nada, solo esperamos  
    }  
}
```

```
    // Combinamos ADRESH (bits altos) y ADRESL (bits bajos)  
    // en un solo valor de 10 bits (0-1023)  
    return ((unsigned int)(ADRESH) << 8) | ADRESL;  
}
```

```
// -----  
// FUNCION PRINCIPAL  
// -----  
void main(void) {
```

```
    unsigned int valorADC;  
    unsigned long milivoltios;
```

```
    TRISD = 0x00;          // PORTD como salida (LCD)  
    TRISAbits.TRISA0 = 1;  // RA0 como entrada (potenciometro)
```

```
    ADC_Inicializar();  
    LCD_Inicializar();
```

```
    while (1) {
```

```
        // Leemos el valor crudo del ADC (0-1023)  
        valorADC = ADC_Leer();
```

```
        // Convertimos el valor ADC a milivoltios reales  
        // Formula: mV = (ValorADC * 5000) / 1023
```

```
milivoltios = ((unsigned long)valorADC * 5000) / 1023;
```

```
// Primera linea: valor crudo del ADC
```

```
LCD_Comando(0x80); // Posicion: linea 1, columna 0
```

```
LCD_EscribirTexto("ADC: ");
```

```
LCD_EscribirNumero(valorADC);
```

```
LCD_EscribirTexto(" "); // Espacios para borrar residuos
```

```
// Segunda linea: voltaje real en milivoltios
```

```
LCD_Comando(0xC0); // Posicion: linea 2, columna 0
```

```
LCD_EscribirTexto("mV: ");
```

```
LCD_EscribirNumero((unsigned int)milivoltios);
```

```
LCD_EscribirTexto(" ");
```

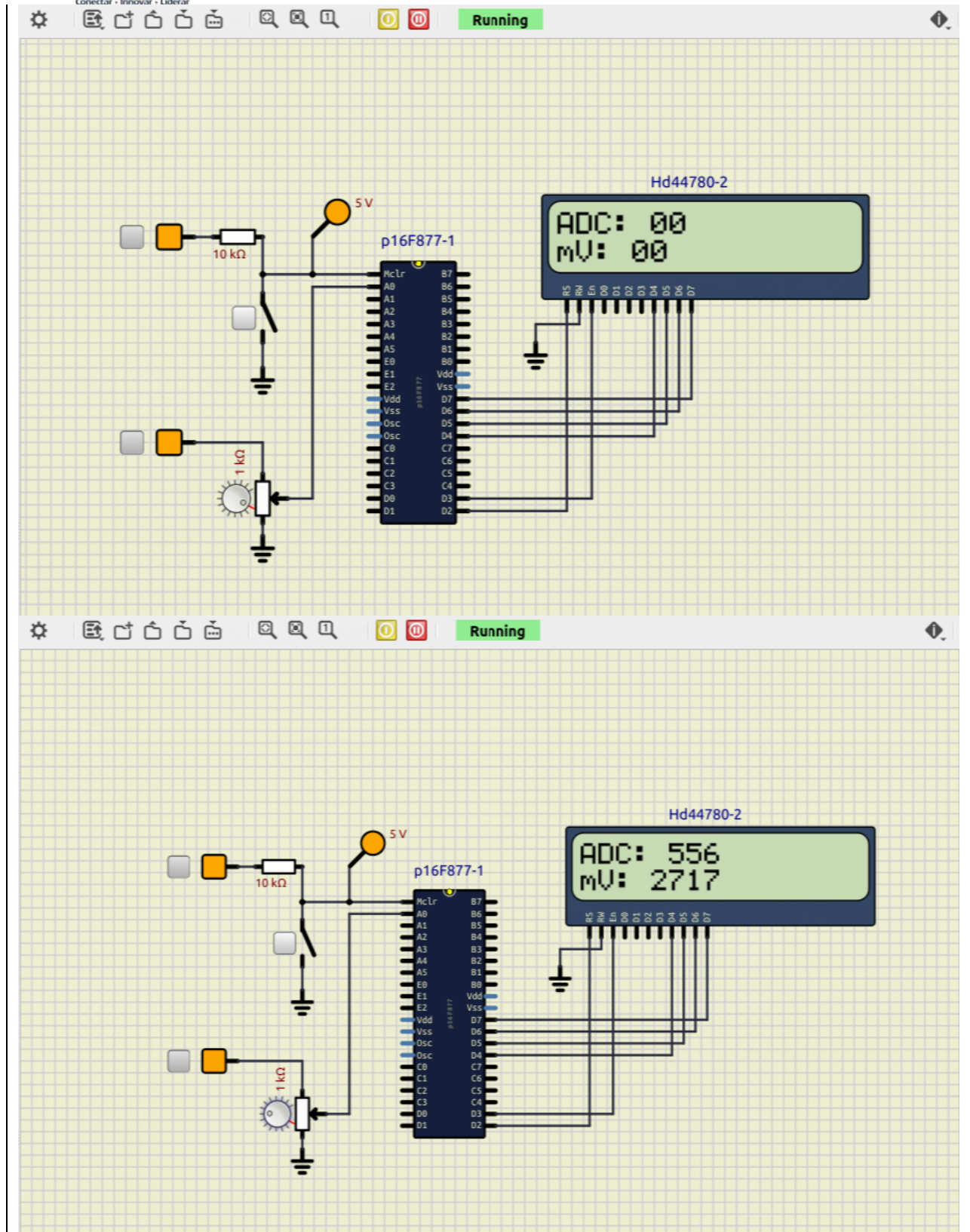
```
__delay_ms(300); // Actualizamos la lectura
```

```
}
```

```
return;
```

```
}
```

Mostrar al docente el funcionamiento en Proteus o SimulID. Pegar aquí el resultado del compilador MPLAB y el simulador.



Concluya sobre el trabajo realizado en este trabajo colaborativo 4

En este trabajo pudimos entender mejor cómo el microcontrolador puede trabajar con señales analógicas utilizando el ADC del PIC16F877A; al variar el potenciómetro pudimos observar cómo el valor leído en RA0 cambia entre 0 y 1023 y cómo ese valor puede convertirse a milivoltios para mostrarlo en el LCD. También nos sirvió para relacionar mejor la parte física de una señal de voltaje con la información que después procesa el programa. En general, esta actividad nos ayudó a comprender de forma más práctica el funcionamiento de las entradas analógicas, la conversión analógico-digital y cómo mostrar los resultados obtenidos en una pantalla LCD.

Valor ____/7 pts. (1 pto. Cada actividad, para un total de 7 pts.)